

# PatchTST

AI for Business Prognosis

Sommersemester 2026

Jasmin Stribik

Islam Abdalla

Fabian Schmid

# PatchTST – Was hat das mit unserem Studium zu tun?



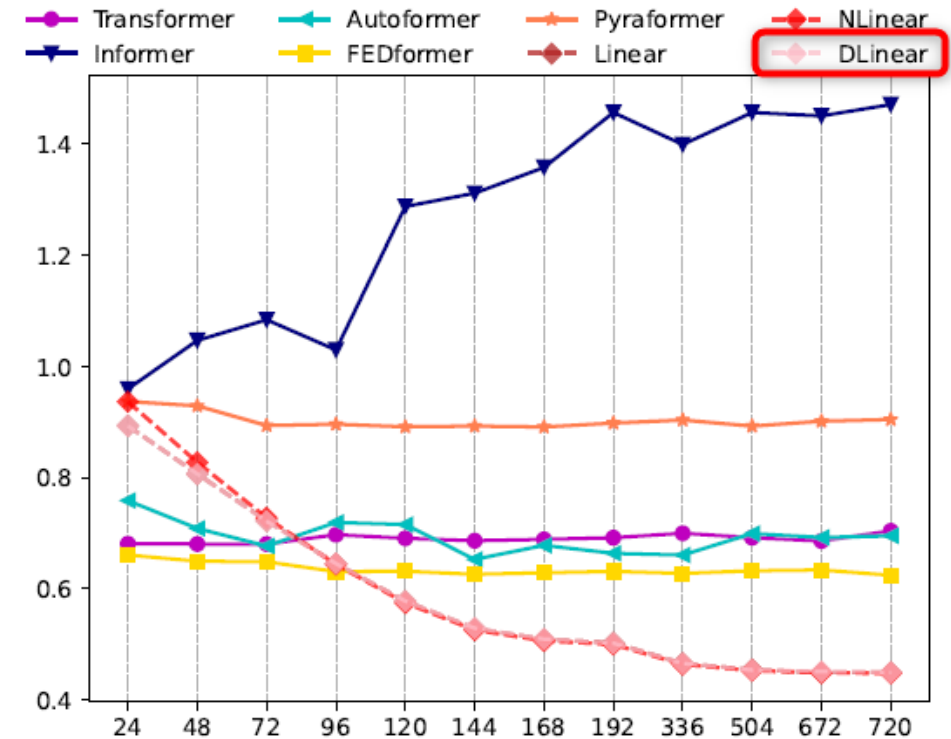
Ein 500-seitiges Skript am Stück zu lernen ist schwierig!

- Mit Karteikarten wird der Stoff in kleine, überschaubare Einheiten zerlegt.
- **PatchTST** macht mit Zeitreihen genau dasselbe

# Zeitreihenvorhersage mit Transformer

- Transformer-basierte Modelle sind in vielen Anwendungen sehr erfolgreich:
  - Sprachverarbeitung (BERT/GPT-Modelle)
  - Bildverarbeitung
- Die besten Ergebnisse bei der Vorhersage von Zeitserien erzielten jedoch bislang andere Modelle:
  - MLP → N-BEATS/N-HITS
  - Lineare Modelle → **DLinear**

Quelle: Peixeiro, 2023 [1]



MSE results (Y-axis) of models with different lookback window sizes (X-axis) of long-term forecasting (T=720) on the Traffic

Quelle: Zeng et al., 2023 [2]

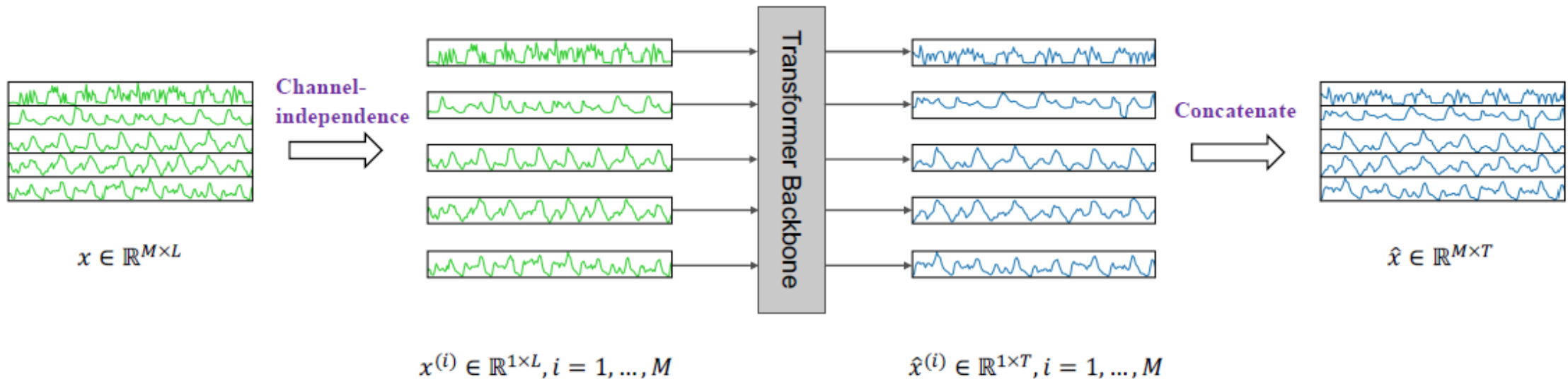
# PatchTST

- Nie et. al: „A TIME SERIES IS WORTH 64 WORDS: LONG-TERM FORECASTING WITH TRANSFORMERS“
- veröffentlicht 2023
- „patch time series transformer“
- Erstes Transformer-Modell, welches Ergebnisse nach dem Stand der Technik bei Zeit-Serien Vorhersagen erzielt

| Models                              | $L$ | $N$ | patch | method          | MSE   |
|-------------------------------------|-----|-----|-------|-----------------|-------|
| Channel-independent<br>PatchTST     | 96  | 96  |       |                 | 0.518 |
|                                     | 380 | 96  |       | down-sampled    | 0.447 |
|                                     | 336 | 336 |       |                 | 0.397 |
|                                     | 336 | 42  | ✓     |                 | 0.367 |
|                                     | 336 | 42  | ✓     | self-supervised | 0.349 |
| Channel-mixing FEDFormer<br>DLinear | 336 | 336 |       |                 | 0.597 |
|                                     | 336 | 336 |       |                 | 0.410 |

A case study of multivariate time series forecasting on Traffic dataset. The prediction horizon is 96. Results with different look-back window  $L$  and number of input tokens  $N$  are reported.  
Quelle: Nie et al., 2023 [3]

# Konzept – High Level



(a) PatchTST Model Overview

## Legende

$M$  = Anzahl der Variablen (Kanäle)  
 $L$  = Länge des Rückblickfensters (Input)  
 $T$  = Länge des Vorhersagehorizonts

$x$  = multivariate Eingabe-Zeitreihe ( $M \times L$ )  
 $x^i$  = einzelne univariate Zeitreihe ( $i = 1 \dots M$ )  
 $\hat{x}$  = prognostizierte Zeitreihe ( $M \times T$ )

# Transformer! – Wir kennen LLMs

- 1** LLM sagt das nächste Wort mit Wahrscheinlichkeiten voraus.

## Beispiel-Prompt



Das Wetter heute ist sehr



LLM  
(Decoder)

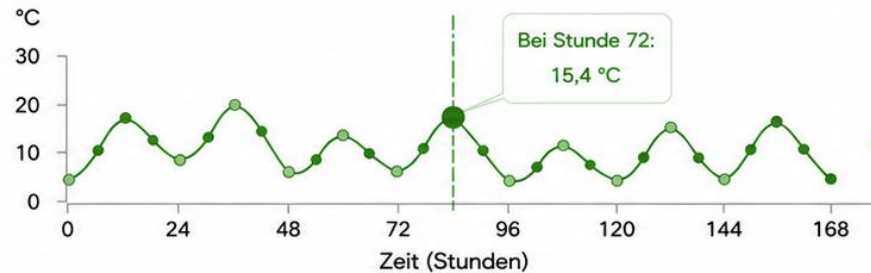
## Wahrscheinlichkeiten für das nächste Wort

|            |             |      |
|------------|-------------|------|
| sonnig     | <div></div> | 0,62 |
| warm       | <div></div> | 0,18 |
| windig     | <div></div> | 0,10 |
| regnerisch | <div></div> | 0,07 |
| ...        |             | ...  |



LLM lernt aus dem Kontext bisheriger Wörter und sagt das nächste Wort mit Wahrscheinlichkeiten voraus.

- 2** Zeitreihe: Ein einzelner Temperaturwert hat wenig Bedeutung



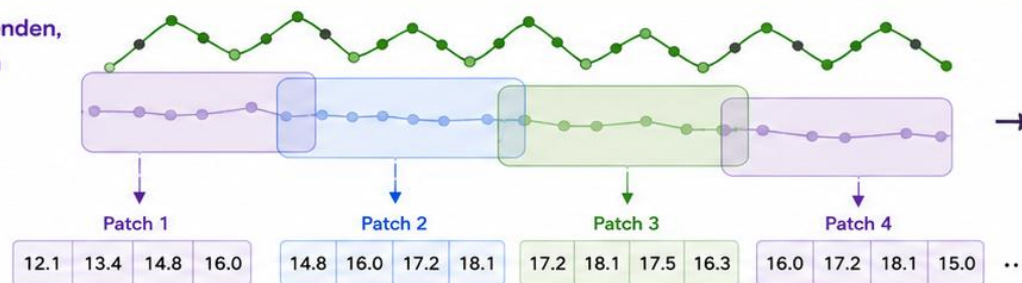
## ? Was sagt uns 15,4 °C?

- × Zu wenig Kontext
- × Keine Trend-Information
- × Keine Saisonalität
- × Schwer, die Zukunft vorherzusagen



Wir brauchen mehr Kontext – Muster über die Zeit – um das wahre Signal zu erkennen.

- 3** Lösung: Patches verwenden, um Muster zu erfassen



## PatchTST (Transformer Encoder)



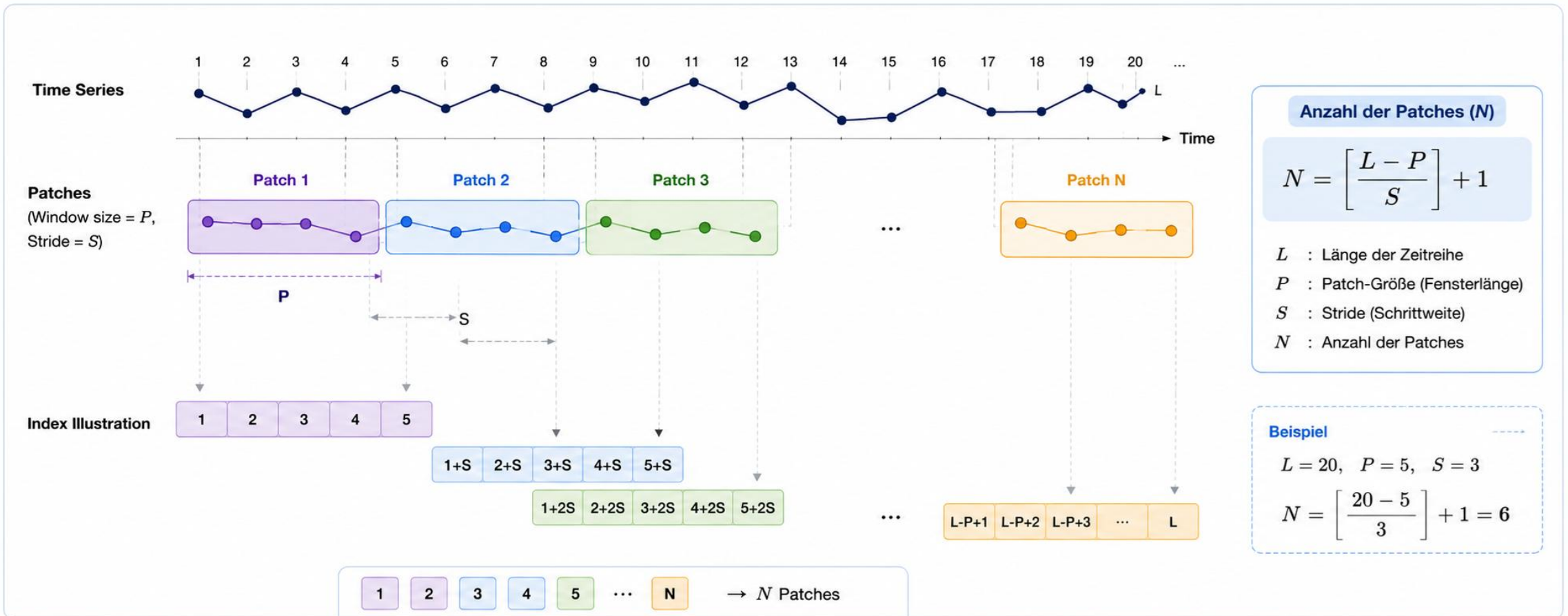
## Prognosehorizont (h)

13.0 12.6 12.2 11.9 ...

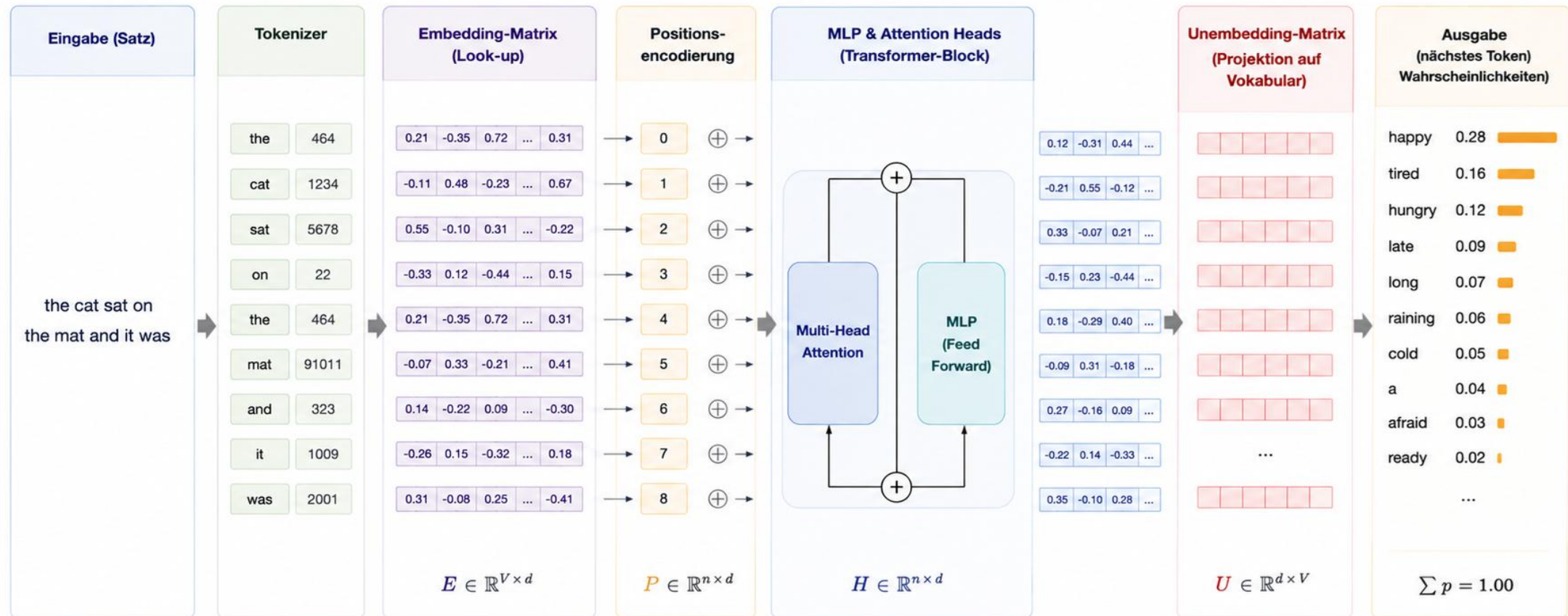
- ✓ Patches erfassen aussagekräftige Muster
- ✓ Transformer modelliert Zusammenhänge
- ✓ Mehrere zukünftige Schritte auf einmal vorhersagen



# Patching – Wie?

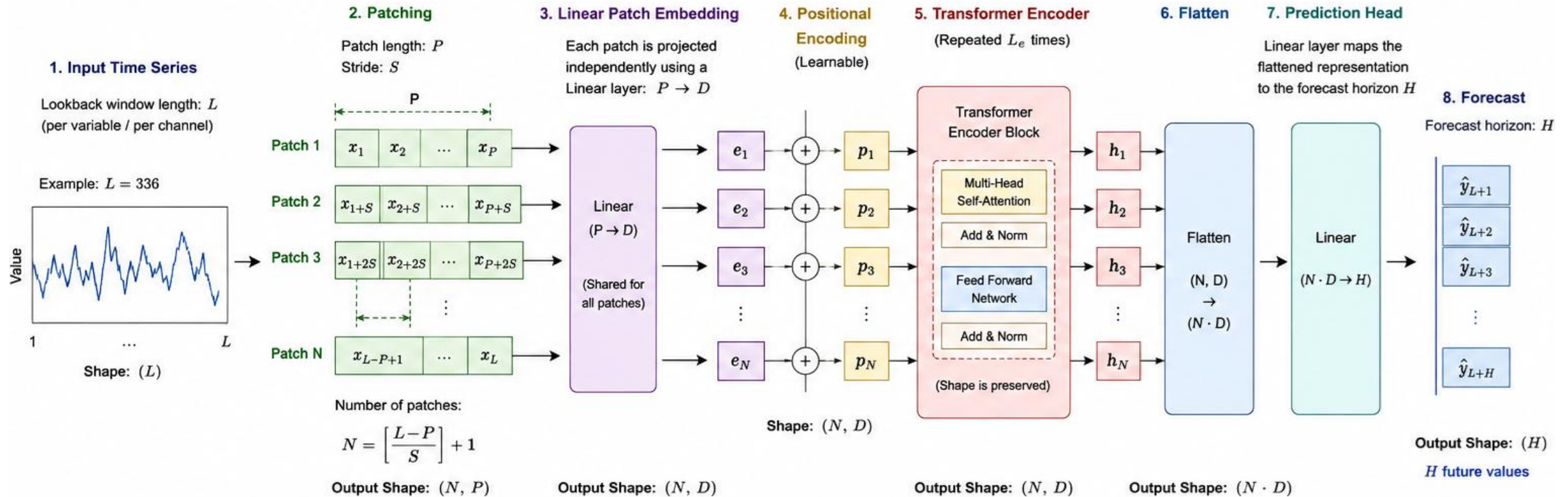


# Transformer! – Wir kennen LLMs

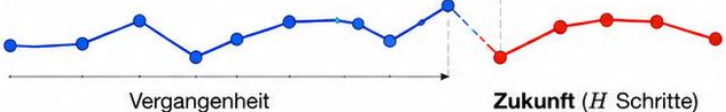




# PatchTST - Architektur

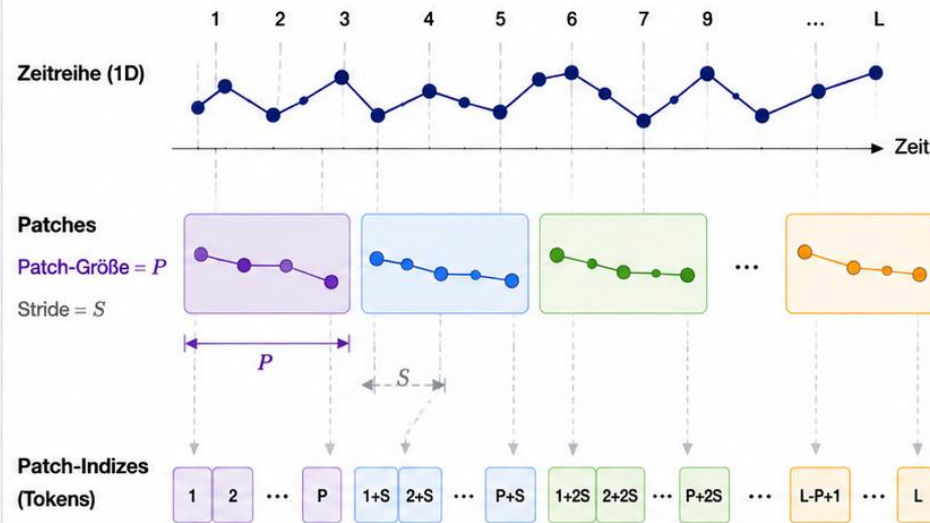


# PatchTST vs LLM

| Aspekt                                                                                                                          | PatchTST (Zeitreihenprognose)                                                                                                                                                                                                                                                                                                                                               | LLM (Sprachmodell)                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>1. Ziel / Aufgabe</b>                      | <b>Zukünftige numerische Werte vorhersagen</b> (Regression)<br> <p>Vergangenheit      Zukunft (<math>H</math> Schritte)</p>                                                                                                                                                               | <b>Nächstes Token (Wort) generieren</b> (Klassifikation)<br><div> <div>The</div> <div>weather</div> <div>is</div> <div>very</div> <div>...</div> <div>sunny</div> <div>?</div> </div> <p>Nächstes Token?</p>                                                                                                                                                                      |
|  <b>2. Eingabe &amp; Tokenisierung</b>         | <b>Kontinuierliche Zeitreihe</b><br><br><b>In überlappende Patches zerlegt</b><br> <p>Patch 1      Patch 2      ...      Patch N</p>                                                                    | <b>Diskretes Token-IDs</b> (Sequenz von Wörtern/Unterwörtern)<br><div> <div>502</div> <div>1215</div> <div>318</div> <div>994</div> <div>...</div> </div> <p>Jedes Token ist bereits diskret</p> <p>Keine Patches.</p>                                                                                                                                                            |
|  <b>3. Embedding</b><br>(Encoding zu Vektoren) | <b>Lineare Projektion jedes Patches</b><br>$P$ (Patch-Länge) $\rightarrow D$ (Embedding-Dimension)<br><div> <div>Patch<br/>(<math>P</math> Werte)<br/>[<math>x_1, x_2, \dots, x_P</math>]</div> <div>→</div> <div>Linear Layer<br/>(<math>W: D \times P</math>)</div> <div>→</div> <div>Embedding<br/>(<math>D</math>)<br/>[<math>e_1, e_2, \dots, e_D</math>]</div> </div> | <b>Lookup im Embedding-Matrix</b><br>$V$ (Vokabulargröße) $\times D$ (Embedding-Dimension)<br><div> <div>Token ID<br/>(z.B. 318)</div> <div>→</div> <div>Embedding Lookup<br/>(Größe: <math>V \times D</math>)</div> <div>→</div> <div>Embedding<br/>(<math>D</math>)<br/>[<math>e_1, e_2, \dots, e_D</math>]</div> </div>                                                        |
|  <b>4. Ausgabe / Vorhersage</b>              | <b>Lineare Projektion auf <math>H</math> zukünftige Werte</b> (Regression)<br><div> <div>Flatten aller<br/>Patch-Embeddings</div> <div>→</div> <div>Linear Layer</div> <div>→</div> <div><math>H</math> Werte<br/>[<math>\hat{y}_1, \hat{y}_2, \dots, \hat{y}_H</math>]</div> </div>                                                                                        | <b>Logits über <math>V</math> Tokens + Softmax</b> (Klassifikation)<br><div> <div>Hidden State</div> <div>→</div> <div>Linear (<math>V</math>)</div> <div>→</div> <div>Softmax</div> <div>→</div> <div>            Wahrscheinlichkeiten<br/>über <math>V</math> Tokens         </div> </div> |

# Kernkonzepte – Was macht PatchTST anders?

## 1. Patching (Zeitliche Segmente)

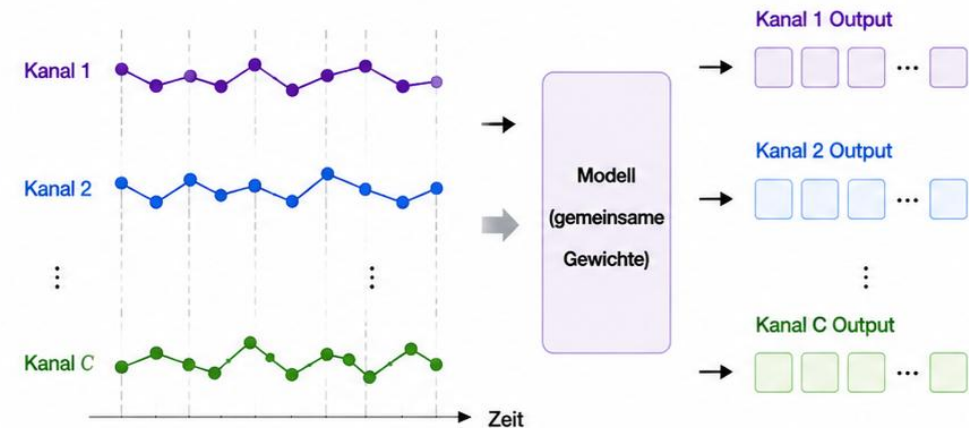


$$\text{Anzahl der Patches } (N) = \left\lceil \frac{L - P}{S} \right\rceil + 1$$

$L$  : Länge der Zeitreihe  
 $P$  : Patch-Größe (Fensterlänge)  
 $S$  : Stride (Schrittweite)  
 $N$  : Anzahl der Patches

## 2. Channel Independence (Kanäle unabhängig verarbeiten)

### Mehrkanalige Zeitreihe



Jeder Kanal wird unabhängig in Patches zerlegt

Gleiches Modell wird auf alle Kanäle angewendet (Gewichte teilen)

Vorhersage pro Kanal (unabhängige Outputs)

# Self-supervised PatchTST

## Zwei Modellvarianten

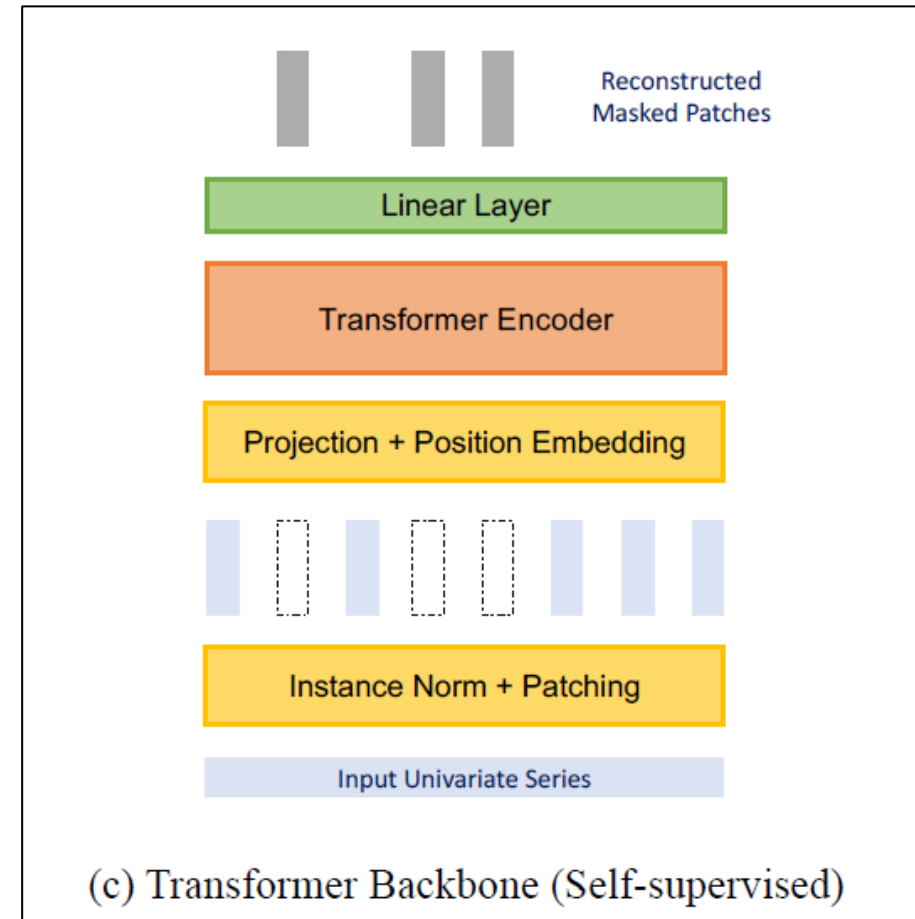
- Supervised PatchTST → klassischer Ansatz
- Self-supervised PatchTST

## Self-supervised

**Ziel:** Allgemeine Zeitreihenrepräsentationen ohne Labels lernen

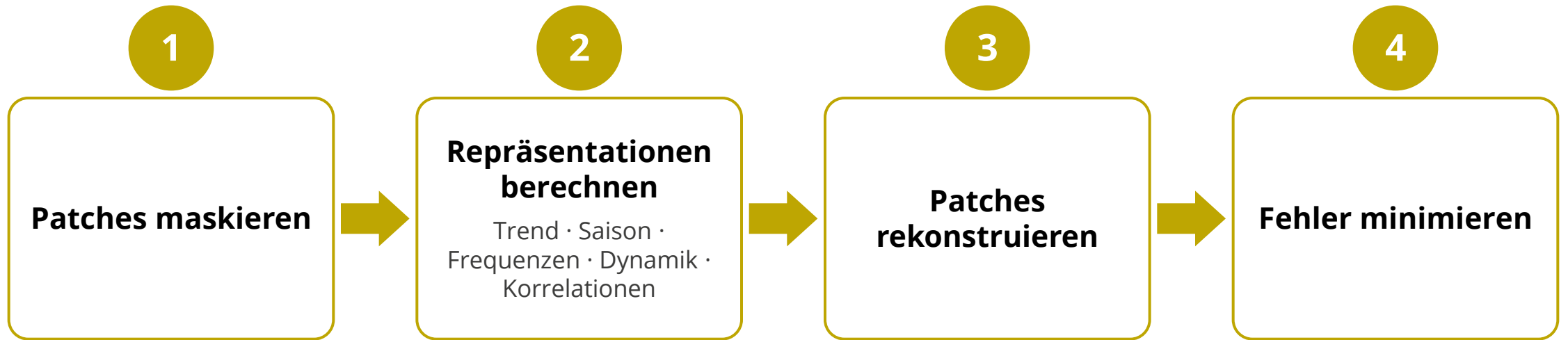
### Training in zwei Schritten:

1. Representation Learning
2. Fine-Tuning: supervised Learning mit gelabelten Daten



Quelle: Nie et al., 2023 [3]

# Ablauf Representation Learning



# Self-supervised PatchTST

Warum funktioniert das noch besser?

- Modell lernt allgemeinem Merkmalsrepräsentationen
- Encoder ist vortrainiert

Was sind die Vorteile?

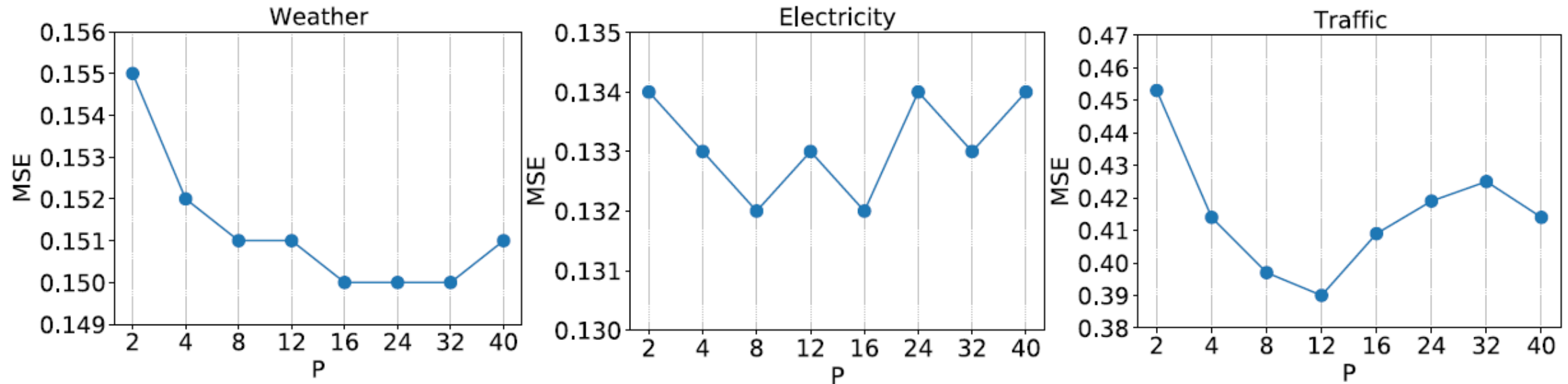
- Kein gelabelter Datensatz für das Vortraining erforderlich
- Bessere Generalisierung
- Höhere Vorhersagegüte insbesondere bei wenig gelabelten Daten

| Models                          | $L$ | $N$ | patch | method          | MSE          |
|---------------------------------|-----|-----|-------|-----------------|--------------|
| Channel-independent<br>PatchTST | 96  | 96  |       |                 | 0.518        |
|                                 | 380 | 96  |       | down-sampled    | 0.447        |
|                                 | 336 | 336 |       |                 | 0.397        |
|                                 | 336 | 42  | ✓     |                 | 0.367        |
|                                 | 336 | 42  | ✓     | self-supervised | <b>0.349</b> |

A case study of multivariate time series forecasting on Traffic dataset. The prediction horizon is 96. Results with different look-back window  $L$  and number of input tokens  $N$  are reported. Quelle: Nie et al., 2023 [3]



# Einfluss der Patchlänge



MSE scores with varying patch lengths  $P = [2, 4, 8, 12, 16, 24, 32, 40]$  without overlapping where the lookback window is 336 and the prediction length is 96.  
Quelle: Nie et al., 2023 [3]

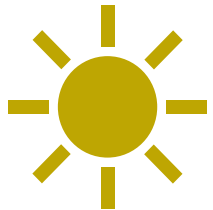
- Keine hohe Varianz → Modell robust gegenüber verschiedenen Patchgrößen
- Je höher P, desto weniger Token → Rechenreduktion
- **Gute Wahl:  $P = \{ 8, 12, 16 \}$**

# Anwendung – Wo funktioniert PatchTST gut und wo nicht?

- **Gut geeignet:**

- Stromverbrauch
- Wetter
- Traffic
- Maschinensensoren
- Einzelne Finanzzeitreihen

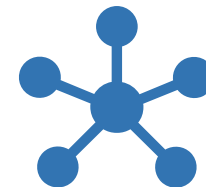
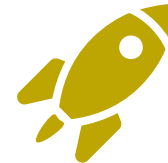
→ Signale mit **individueller** zeitlicher Dynamik



- **Weniger gut geeignet:**

- Gekoppelte physikalische Systeme
- Chemische Prozesse
- Graph-/Netzwerkdaten
- Regelungstechnik

→ Wenn **Wechselwirkung** zwischen Signalen entscheidend für Vorhersage



→ **Channel-Independence ist Stärke und Limitierung gleichzeitig**

# Implementierung

- **Manuell anhand des Papers (from scratch):**
  - Volle Kontrolle & Flexibilität, aber hoher Implementierungsaufwand
  - Kernbausteine: Patching → Channel-Independence → Transformer-Encoder → linearer Head
  - Referenz: offizielles Repository ([github.com/yuqinie98/PatchTST](https://github.com/yuqinie98/PatchTST))
- **Bibliotheken (High-Level, wenige Zeilen Code):**
  - **Hugging Face Transformers** – PatchTSTForPrediction; vortrainierte Modelle (IBM Granite), Fine-Tuning  
`model = PatchTSTForPrediction.from_pretrained(model_id)`
  - **NeuralForecast (Nixtla)** – High-Level-API, schnellster Einstieg  
`nf = NeuralForecast(models=[PatchTST(h=24, input_size=96)])`
  - **GluonTS / Time-Series-Library** – weitere Frameworks für Forschung & Benchmarks

# Key-Learnings

- PatchTST nutzt Patches zur Vorhersage
- Das Modell PatchTST setzt auf Channel-Independence
- Vorteile: geringere Rechenzeit
- Anwendungsfälle:
  - Energiedaten
  - Finanzdaten
  - Wetter

# Live-Demo

# Quellen

1. Peixeiro, M. (2023): PatchTST: A Breakthrough in Time Series Forecasting. Medium: <https://medium.com/the-forecaster/patchtst-a-breakthrough-in-time-series-forecasting-e02d48869ccc>
2. Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023): Are Transformers Effective for Time Series Forecasting? AAAI 2023. arXiv:2205.13504
3. Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023): A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR 2023. arXiv:2211.14730